

Encoding-dispatch Migration Notes

clautolisp

June 24, 2026

Contents

Scope

This document explains, for users coming from AutoCAD or BricsCAD, how the file-encoding extensions they're used to map into clautolisp. Read the section matching the vendor whose codebase you're porting from; the parallel section for the other vendor is documented for cross-reference.

The full normative reference is `issues/closed/encoding-dispatch.issue` (the cross-dialect translation table plus the per-dialect dispatch matrix). The user-manual section **Encoding-dispatch diagnostics (enc-)*** documents the diagnostic codes and the two suppression knobs (per-code pragma + `--lax`).

Quick reference

Concept	AutoCAD form	BricsCAD form	clautolisp form
Default file encoding	(none — global LISPSYS)	(none — per-file BOM sniff)	<code>*AUTOLISP</code>
Per-call encoding on <code>open</code>	(<code>open f m "utf8"</code>)	(<code>open f "MODE,ccs=ENC"</code>)	(<code>open f m</code>)
Per-call encoding on <code>load</code>	not available	not available	(<code>load f m</code>)
Detect a file's encoding	not available	(<code>vle-file-encoding f</code>)	(<code>clal-f-encoding f</code>)
Host code page (read-only)	(<code>getvar "SYSCODEPAGE"</code>)	(<code>getvar "SYSCODEPAGE"</code>)	(<code>clal-host-codepage</code>)
Drawing code page (read-only)	(<code>getvar "DWGCODEPAGE"</code>)	(<code>getvar "DWGCODEPAGE"</code>)	(<code>clal-dwg-codepage</code>)
Detect codepage mismatch	manual comparison	manual comparison	(<code>clal-codepage-mismatch</code>)
Engine version gate	(<code>getvar "LISPSYS"</code>)	not present	<code>*AUTOLISP</code>

Coming from AutoCAD

Source-file encoding selection

AutoCAD: gated by LISPSYS (a host-side system variable):

```
;; AutoCAD 2021+:  
(setvar "LISPSYS" 1)           ; switch to Unicode engine  
(load "src/foo.lsp")
```

clautolisp: use the `*AUTOLISP-FILE-ENCODING*` global, which is read-write and per-session — or pass the encoding directly on each load call:

```
;; clautolisp:  
(setq *AUTOLISP-FILE-ENCODING* "UTF-8")  
(load "src/foo.lsp")
```

```
;; or per-call, the clautolisp-only extension:  
(load "src/foo.lsp" nil "UTF-8")
```

The `*AUTOLISP-FILE-ENCODING*` global is the spec-mandated, more expressive replacement: it lets the encoding change between LOAD calls without binding a sysvar that affects the whole engine.

Under `--autocad` the clautolisp load-with-encoding form emits `enc-foreign-dialect`. To silence it for a portable section, use the pragma:

```
(clal-suppress-enc-diagnostic 'enc-foreign-dialect)  
(load "src/encoding-aware.lsp" nil "UTF-8")  
(clal-enable-enc-diagnostic 'enc-foreign-dialect)
```

open third argument

AutoCAD: `(open f "r" "utf8")` or `(open f "r" "utf8-bom")`.

clautolisp: accepts the same literals AND the broader clautolisp vocabulary (UTF-8-BOM / ISO-8859-1 / CP-NNNN / ...). The vendor literals are silent under `--autocad`; broader spellings emit `enc-foreign-dialect`.

```
;; Both forms work in clautolisp under --autocad:  
(open "data.txt" "r" "utf8")           ; AutoCAD literal, silent  
(open "data.txt" "r" "UTF-8")         ; clautolisp form, foreign-dialect
```

LISPSYS system variable

AutoCAD: LISPSYS values are 0 (legacy MBCS), 1, or 2 (Unicode).

clautolisp: LISPSYS is exposed via the catalogue; setvar validates against {0, 1, 2} and emits `enc-lispsys-out-of-range` if outside (the write still proceeds). Under `--bricscad` / `--clautolisp` / `--strict`, even reading LISPSYS emits `enc-foreign-dialect` — AutoCAD-only sysvar.

```
;; OK under --autocad - silent:
(setvar "LISPSYS" 1)

;; Same call under --strict:
;; [enc-extension-used] SETVAR on LISPSYS: AutoCAD-only sysvar, foreign to --strict.

;; Out-of-range, any dialect:
(setvar "LISPSYS" 7)
;; [enc-lispsys-out-of-range] (setvar "LISPSYS" 7): valid values are 0, 1, or 2.
```

Host code page

AutoCAD: (getvar "SYSCODEPAGE") returns "ANSI_1252"-style strings.

clautolisp: (getvar "SYSCODEPAGE") is preserved (vendor compatibility). The new (clal-system-codepage) helper returns the canonical clautolisp spelling "CP-1252" / "UTF-8" etc., collapsing vendor ANSI_{NNNN} and Unicode-name aliases onto a single form.

```
(getvar "SYSCODEPAGE")      ; => "ANSI_1252" (vendor format)
(clal-system-codepage)      ; => "CP-1252"   (clautolisp canonical)
```

Coming from BricsCAD

Source-file encoding selection

BricsCAD: load sniffs the BOM per-file.

clautolisp: also BOM-aware via the new (clal-file-encoding PATH) helper. To explicitly drive a load with a known encoding, use the clautolisp positional [encoding] form (under `--bricscad` this emits `enc-foreign-dialect`):

```
;; Detect a file's BOM the BricsCAD way:
(vle-file-encoding "src/data.txt") ; => "UTF-8" (BricsCAD)
(clal-file-encoding "src/data.txt") ; => "UTF-8-BOM" (clautolisp,
```

```
; preserves the BOM/no-BOM
; distinction)
```

open ,ccs=ENC mode suffix

BricsCAD: (open f "r,ccs=UTF-8"), (open f "w,ccs=UNICODE") (where UNICODE is an alias for UTF-16-LE).

clautolisp: accepts the suffix syntax; the mode-string splitter strips the ,ccs=ENC tail before CL OPEN sees it. Under `--bricscad` the form is silent (native); under `--clautolisp` / `--autocad` it emits `enc-foreign-dialect`.

Idiomatic clautolisp uses the positional third argument instead:

```
;; BricsCAD form (works in clautolisp; under --clautolisp it warns):
(open "data.txt" "w,ccs=UTF-8")
```

```
;; clautolisp form (the recommended spelling):
(open "data.txt" "w" "UTF-8")
```

vle-file-encoding

BricsCAD: (vle-file-encoding f) returns one of "ANSI" / "UTF-8" / "UTF-16LE" / "UTF-16BE" / "UTF-32LE" / "UTF-32BE".

clautolisp: (clal-file-encoding f) returns canonical clautolisp strings: "UTF-8-BOM" (preserving the BOM/no-BOM distinction BricsCAD flattens), "UTF-16-LE" (with the hyphen matching clautolisp's vocabulary), "ANSI" for no-BOM files, or `nil` when the path doesn't resolve.

Host code page

Identical to AutoCAD's. BricsCAD exposes `SYSCODEPAGE` / `DWGCODEPAGE` with the same semantics and identical name. The clautolisp `clal-system-codepage` / `clal-drawing-codepage` helpers canonicalise both equally.

Suppressing diagnostics during migration

When porting a vendor codebase, the runtime diagnostics surface every encoding extension the code uses. Three ways to silence them:

1. **Switch the dialect** matching the source's vendor — e.g. `clautolisp` `--autocad` for AutoCAD code, `--bricscad` for BricsCAD code. The native form is silent under its own dialect.

2. **Per-code pragma** at the top of the file:

```
(clal-suppress-enc-diagnostic 'enc-foreign-dialect
                             'enc-extension-used)
;; the rest of the file silences those two codes
```

3. **-lax dialect** for catch-all suppression:

```
clautolisp --lax src/heterogeneous-codebase.lsp
```

--lax silences every encoding diagnostic globally; the runtime still honours each form.

CI-gate for encoding extensions

To catch encoding extensions statically before they ship — even in code paths the tests don’t exercise — pipe a file’s parsed form-tree through the lint:

```
(dolist (form (read-source-forms "src/foo.lsp"))
  (clal-lint-encoding-extensions form))
```

The lint emits the same **enc-*** diagnostics the runtime would. Run it under your target dialect (e.g. **--strict** when you want zero extensions to ship) and gate the build on whether anything emits.

Source notes

This document is part of the encoding-dispatch ship. See:

- [issues/closed/encoding-dispatch.issue](#) — the full cross-dialect translation table + dispatch matrix.
- The user manual section **Encoding-dispatch diagnostics (enc-)*** — the eight diagnostic codes.
- The autolisp-spec function entries for **CLAL-SYSTEM-CODEPAGE** / **CLAL-DRAWING-CODEPAGE** / **CLAL-CODEPAGE-MISMATCH-P** / **CLAL-FILE-ENCODING** — normative reference.